arXiv:2602.10408v1 [cs.LG] 11 Feb 2026

Gated Removal of Normalization in Transformers Enables Stable Training and Efficient Inference

Andrei Kanavalau [1] **Carmen Amo Alonso** [2] **Sanjay Lall** [1]

Abstract

Normalization is widely viewed as essential for stabilizing Transformer training. We revisit this assumption for pre-norm Transformers and ask to what extent sample-dependent normalization is needed inside Transformer blocks. We introduce *TaperNorm*, a drop-in replacement for RMSNorm/LayerNorm that behaves exactly like the standard normalizer early in training and then smoothly tapers to a learned sample-independent linear/affine map. A single global gate is held at $g{=}1$ during gate warmup, used to calibrate the scaling branch via EMAs, and then cosine-decayed to $g{=}0$, at which point per-token statistics vanish and the resulting fixed scalings can be folded into adjacent linear projections. Our theoretical and empirical results isolate scale anchoring as the key role played by output normalization: as a (near) 0-homogeneous map it removes radial gradients at the output, whereas without such an anchor cross-entropy encourages unbounded logit growth ("logit chasing"). We further show that a simple fixed-target auxiliary loss on the pre-logit residual-stream scale provides an explicit alternative anchor and can aid removal of the final normalization layer. Empirically, TaperNorm matches normalized baselines under identical setups while eliminating per-token statistics and enabling these layers to be folded into adjacent linear projections at inference. On an efficiency microbenchmark, folding internal scalings yields up to $1.22\times$ higher throughput in last-token logits mode. These results take a step towards norm-free Transformers while identifying the special role output normalization plays.

[1]Department of Electrical Engineering, Stanford University, Stanford, USA [2]Department of Computer Science, Stanford University, Stanford, USA. Correspondence to: Andrei Kanavalau <kanaval@stanford.edu>.

Preprint. February 12, 2026.

1. Introduction

Normalization layers combined with residual connections were instrumental in making very deep networks trainable. Residual paths preserve gradients (He et al., 2015) while normalization layers such as BatchNorm, LayerNorm, and RMSNorm stabilize activations (Ioffe & Szegedy, 2015; Ba et al., 2016; Zhang & Sennrich, 2019). The latter two are commonly used in Transformer based language models where normalization is applied per-token. Per-token normalization introduces a data-dependent nonlinearity into every sublayer and incurs a computational cost because per-token statistics must be computed. It can also hinder mechanistic interpretability because it obscures the meaning of the residual-stream and makes it harder to decompose models into circuits (Baroni et al., 2025). This work examines whether per-token normalization in every Transformer block is necessary. The aim is to exploit normalization during the unstable early phase of training while ultimately producing a model that is norm-free and therefore simpler and cheaper at inference.

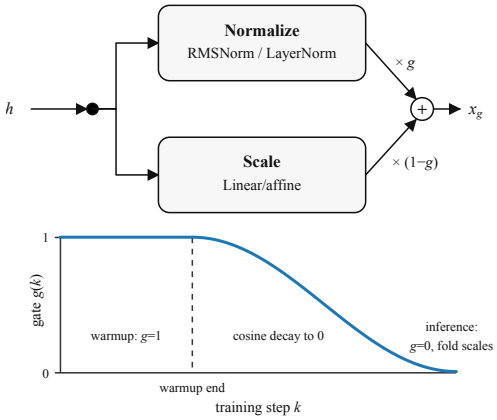

Figure 1. TaperNorm layer and gate scheduling.

To this end we introduce a gated layer, *TaperNorm*, that begins as ordinary RMSNorm/LayerNorm and then tapers to a simple sample-independent linear/affine map. A single global gate controls TaperNorm behavior such that early in training the model behaves exactly like a pre-norm Transformer. As the gate decays, normalizers are smoothly replaced by sample-independent scaling. At convergence,

per-token statistics are no longer used in the tapered layers. In our default setting we keep the final normalization active as an implicit scale anchor, but we also show that an explicit scale anchor, namely a fixed-target auxiliary loss on the pre-logit residual stream, can replace the final normalization. This enables all normalization layers to be tapered away.

Our mathematical analysis clarifies that what governs these behaviors is scale anchoring at the output. A final normalization layer is a convenient implicit anchor that removes radial gradients at the output, so the loss cannot be reduced by simply inflating the last hidden-state norm. Without such an anchor, cross-entropy has a radial component that drives logit magnitudes upward ("logit chasing"). We show that the fixed-target auxiliary loss introduces a counteracting radial restoring force, explaining why it can suppress logit chasing even when the final normalization is removed.

Removing per-token normalization is useful primarily at inference time. It has the potential to reduce computational costs of inference and aid interpretability efforts. In our approach, at the end of training each TaperNorm becomes a fixed linear or affine map, so its effect can be absorbed into adjacent linear projections (e.g. attention and MLP input projections).

Our key contributions are:

- A drop-in gated layer that transitions from tokenwise normalization to a fixed linear or affine rescaling, enabling removal of per-token statistics and operator fusion at inference.
- Analysis of pre-norm Transformers in which the final normalization acts as an implicit scale anchor that removes radial gradients, cross-entropy induces logit chasing without an anchor, and a fixed-target scale loss provides an explicit restoring force.
- Experiments on TinyStories and GPT-2 showing competitive loss while tapering normalization, and up to $1.22\times$ inference throughput improvements.

2. Related Work

Normalization has been central to stabilizing deep networks. BatchNorm improves optimization by normalizing intermediate activations (Ioffe & Szegedy, 2015), while LayerNorm removes batch coupling and became standard in sequence models (Ba et al., 2016). In Transformers, the pre-norm configuration, where input to each sublayer is normalized, improves stability over the original post-norm, where the residual stream is normalized after every sublayer contribution (Xiong et al., 2020). RMSNorm retains scale normalization but omits mean subtraction (Zhang & Sennrich, 2019) achieving a reduction in computational cost.

Two lines of work reduce reliance on per-token normalization. One replaces it via special initialization or residual scaling—Fixup (Zhang et al., 2019), ReZero (Bachlechner et al., 2020), LayerScale (Touvron et al., 2021), and DeepNorm (Wang et al., 2022). The other pursues normalizer-free training by modifying activations or attention: NF-Nets for ConvNets (Brock et al., 2021); Deep Kernel Shaping and related transformer modifications (Martens et al., 2021; He et al., 2023); and dynamic element-wise nonlinearities such as DyT (Zhu et al., 2025). Our approach takes a different route: we keep standard pre-norm dynamics when optimization is most unstable, then taper internal normalizers to a fixed linear/affine rescaling learned from data. By default we retain the final normalization as a convenient implicit scale anchor, and we additionally show that a fixed-target scale loss can serve as an explicit anchor that helps stabilize the removal process and also enables tapering away the final normalization as well when internal normalization is removed.

Closest in spirit is the work by Baroni et al. (2025) who show that LayerNorm layers in GPT-2 can be removed with a staged procedure and small accuracy loss. We complement this with a single global taper that avoids staged layer-by-layer removal schedules and directly targets convenient removal of normalization layers.

Compared to norm-replacement nonlinearities such as DyT (Zhu et al., 2025), our taper converges to a linear or affine map that can be absorbed into adjacent projections and removed from the inference graph. Compared to residual-scaling or initialization-based approaches (e.g. Fixup, ReZero, LayerScale, DeepNorm), our mechanism targets the normalization operator directly while keeping standard pre-norm behavior during the early phase. Finally, relative to staged LayerNorm removal in GPT-2 (Baroni et al., 2025), the single global taper avoids sequential layer schedules and is applicable to model pre-training.

3. Method

We study pre-norm Transformers and introduce a dynamic normalization layer *TaperNorm* that enables standard normalized dynamics during early training while enabling inference with fewer normalization operations. TaperNorm is a drop-in replacement for tokenwise RMSNorm or LayerNorm. It computes a convex combination of a standard normalization branch and a sample-independent scaling branch. A single scalar gate value $g(k) \in [0, 1]$ shared across layers and tokens at training step k interpolates between the branches. We keep $g(k) = 1$ during an initial gate warmup period so the model matches the normalized baseline while we accumulate exponential moving average (EMA) statistics. At the end of this period (the taper start), each TaperNorm layer sets a per-layer coefficient c from

these EMAs, copies per-feature scales from the normalization branch, and then freezes c. After taper start, $g(k)$ is cosine-decayed until $g = 0$. When $g = 0$ the layer becomes linear/affine, so it can be folded into adjacent linear projections for inference.

3.1. Pre-norm Transformer and notation

We consider models with embedding size d and sequence length T. Let $H^{(0,0)} \in \mathbb{R}^{T\times d}$ denote the input to the first block. Each layer j has multi-head self-attention (MHSA) and a feedforward network (MLP). Writing $\mathrm{Attn}(H;\theta_{j,\mathrm{A}})$ and $\mathrm{MLP}(H;\theta_{j,\mathrm{M}})$ for these components, the pre-norm residual updates are

$$\begin{aligned} H^{(j,1)} &= H^{(j,0)} + \mathrm{Attn}\Big(\mathrm{Norm}_j^{\mathrm{A}}\big(H^{(j,0)}\big);\ \theta_{j,\mathrm{A}}\Big), \\ H^{(j,2)} &= H^{(j,1)} + \mathrm{MLP}\Big(\mathrm{Norm}_j^{\mathrm{M}}\big(H^{(j,1)}\big);\ \theta_{j,\mathrm{M}}\Big), \\ H^{(j+1,0)} &= H^{(j,2)}, \end{aligned}$$

for $j = 0, ..., L-1$. In the TinyStories pre-training experiments we use SwiGLU activations and RoPE positional encoding.

After the L layers, we apply an output map $\mathrm{Norm}_{\mathrm{final}}$ tokenwise. In our main experiments $\mathrm{Norm}_{\mathrm{final}}$ is a standard RMSNorm or LayerNorm. In ablations that remove output normalization, we implement $\mathrm{Norm}_{\mathrm{final}}$ as a tapered layer whose gate is decayed to $g = 0$ over training. We represent token vectors as row vectors, so linear maps act on the right. A learned projection $W_{\mathrm{out}} \in \mathbb{R}^{d\times V}$ yields logits

$$z \;=\; \mathrm{Norm}_{\mathrm{final}}\big(H^{(L,0)}\big)\, W_{\mathrm{out}} \in \mathbb{R}^{T\times V}.$$

For a token at position t, let $z_t \in \mathbb{R}^V$ denote the t-th row of z and let y_t be its target id. The token-level cross-entropy is $\ell(z_t, y_t) = -\log \mathrm{softmax}(z_t)_{y_t}$.

3.2. LayerNorm and RMSNorm

Given token row vector $h \in \mathbb{R}^{1\times d}$, LayerNorm and RMSNorm are

$$\begin{aligned} \mathrm{LayerNorm}(h) &= \frac{h-\mu_h}{\sigma_h}\, D_\gamma + \beta, \\ \mu_h = \tfrac{1}{d}\sum_{i=1}^{d} h_i, \quad \sigma_h &= \sqrt{\tfrac{1}{d}\sum_{i=1}^{d}(h_i-\mu_h)^2 + \varepsilon}, \end{aligned} \tag{1}$$

$$\begin{aligned} \mathrm{RMSNorm}(h) &= \frac{h}{r(h)}\, D_\gamma, \\ r(h) &= \sqrt{\|h\|_2^2/d + \varepsilon}, \end{aligned} \tag{2}$$

with learnable per-feature scales $\gamma \in \mathbb{R}^d$ (and bias $\beta \in \mathbb{R}^d$ for LayerNorm). We write $D_\gamma \coloneqq \mathrm{diag}(\gamma)$ and $D_{\tilde\gamma} \coloneqq \mathrm{diag}(\tilde\gamma)$ for diagonal scalings.

3.3. TaperNorm

TaperNorm dynamically transitions from RMSNorm/LayerNorm to a fixed linear or affine map of features. For gate $g \in [0,1]$ and token $h \in \mathbb{R}^{1\times d}$,

$$\mathrm{TaperNorm}(h;g) \;=\; g\,\frac{h}{r(h)}\, D_\gamma \;+\; (1-g)\, c\, h\, D_{\tilde\gamma}, \tag{3}$$

where $r(h)$ is as in RMSNorm, $c \in \mathbb{R}$ is a per-layer scalar, and $\tilde\gamma \in \mathbb{R}^d$ is a per-feature gain that remains trainable during tapering. The LayerNorm variant, used in GPT-2 experiments, is analogous and can be found in Appendix B. For $g = 1$ we recover RMSNorm/LayerNorm. For $g = 0$ we obtain a sample-independent scaling that can be fused into downstream linear operations.

We keep $g(k) = 1$ during an initial gate warmup period to obtain standard normalized dynamics and to accumulate EMA statistics used to set c. At the end of gate warmup (the taper start), each TaperNorm layer computes its own $c^\star$ from γ-weighted EMAs (Section 3.4), freezes this scalar for the remainder of training, and initializes $\tilde\gamma \leftarrow \gamma$. Both γ and $\tilde\gamma$ remain trainable and continue to update until convergence. After taper start, $g(k)$ is cosine-decayed from 1 to 0. This same scalar $g(k)$ is applied to every TaperNorm layer that is being tapered. At convergence, $g = 0$ and per-token statistics are no longer used.

When $g = 0$, TaperNorm can be removed from the inference graph by weight folding. For the RMSNorm case, $\mathrm{TaperNorm}(h;0) = c\, h\, D_{\tilde\gamma}$. If the next operation is a linear map $x \mapsto xW$ (with an optional bias), then we can replace W by $c\, D_{\tilde\gamma} W$ and remove the explicit TaperNorm. The same folding applies to attention and MLP input projections. For the LayerNorm variant, the $g = 0$ map is affine and can be folded into a following linear layer as described in Appendix B.

TaperNorm can be used to replace any subset of normalization layers. In this work we primarily taper internal normalizers, since this yields major inference savings (Section 5.2) while remaining stable across runs. Tapering the final norm provides little additional speedup. We report All-Taper ablations to demonstrate fully norm-free inference and because this regime may be of interest beyond inference efficiency e.g. mechanistic interpretability.

3.4. Calibrating the scaling branch

TaperNorm interpolates between a normalized branch and a sample-independent scaling branch. When the gate begins to decay after warmup, we would like the two branches to agree as closely as possible so the interpolation does not introduce an avoidable distribution shift. We therefore align the scaling branch to the normalization branch at the warmup boundary by setting $D_{\tilde\gamma} \leftarrow D_\gamma$ and choosing a